

Abaqus Substructures (Superelements) — What, Why, and How

A working reference for substructuring, grounded in the actual assembly at hand: **6 identical tubesheets and 6 identical spools** in a global model, run **linear static**.

1. What a substructure is

A **substructure** (or **superelement**) is a piece of a model that has been mathematically reduced to a small **stiffness matrix** relating only its **retained (boundary) degrees of freedom**. Every interior DOF is condensed out. The reduced part then behaves, in a larger model, like a single custom element — a "complex spring" whose stiffness exactly matches the detailed part it replaces (within the mesh).

Think of it as **once** solving the hard question "how does this part respond at its interfaces?" and then reusing that answer everywhere the part appears.

The reduction, in one equation

Partition the part's full stiffness into **retained** DOF ($\mathbf{r}$), the interface) and **internal** DOF ($\mathbf{i}$, everything else):

$$\begin{bmatrix} \mathbf{K}_{rr} & \mathbf{K}_{ri} \\ \mathbf{K}_{ir} & \mathbf{K}_{ii} \end{bmatrix} \begin{bmatrix} \mathbf{u}_r \\ \mathbf{u}_i \end{bmatrix} = \begin{bmatrix} \mathbf{f}_r \\ \mathbf{f}_i \end{bmatrix}$$

With no loads on the interior ($\mathbf{f}_i = \mathbf{0}$), the second row gives $\mathbf{u}_i = -\mathbf{K}_{ii}^{-1} \mathbf{K}_{ir} \mathbf{u}_r$. Substituting into the first row:

$$\mathbf{K}_{red} = \mathbf{K}_{rr} - \mathbf{K}_{ri} \mathbf{K}_{ii}^{-1} \mathbf{K}_{ir} \quad (\text{Guyan / static condensation})$$

$\mathbf{K}_{red}$ is the **reduced stiffness matrix** — small, dense, and exact for linear static behavior. For our parts, with two interfaces each lumped to a 6-DOF reference node, $\mathbf{K}_{red}$ is 12×12 .

The term $\mathbf{K}_{ii}^{-1} \mathbf{K}_{ir}$ is the expensive part (it factorizes the whole interior). It is computed **once** at generation and stored as the **recovery matrix**, which later reconstructs interior response from interface motion.

2. Why use one — where the efficiency actually comes from

The efficiency is **not** in a single solve. A one-off part is often cheaper to just mesh directly.

The gains appear under **reuse and repetition**:

2.1 Reuse of an identical part (this is our case)

The 6 tubesheets are identical, and the 6 spools are identical. So you:

- **Generate the tubesheet superelement once**, place it **6 times**.
- **Generate the spool superelement once**, place it **6 times**.

The costly interior factorization (K_{ii}^{-1}) happens **twice total** — once per unique part — instead of once per instance per solve. The global model then carries $6 \times 12 + 6 \times 12 = 144$ reduced DOF for all twelve parts, versus **millions** of DOF if every hole and wave slit in every instance were meshed.

2.2 Repetition across load cases

In the global model, each additional load case re-solves only the small reduced system. The interior factorization is never repeated. Ten load cases → the perforated interiors are still factorized only twice, not twenty times.

2.3 Model-size reduction

A single detailed tubesheet (holes meshed) can be hundreds of thousands to millions of DOF. Six of them plus six spools, meshed in full, is likely intractable to assemble and solve as one model. Reduced to superelements, the global assembly is small enough to solve quickly and iterate on.

2.4 Practical / workflow gains

- **Separation of concerns.** The detailed part is verified once, in isolation (mesh convergence, hole stress), then frozen as a superelement. The global analyst cannot accidentally disturb it.
- **IP / sharing.** A superelement can be handed off without exposing the detailed mesh.
- **Faster iteration.** Global load cases, BCs, and neighboring parts can change without re-meshing or re-reducing the tubesheet or spool.

The one-line intuition: substructuring pays off when the same expensive interior is solved **more than once** — across instances, across load cases, or both. With 6 identical parts × several load cases, that condition is strongly met.

3. The stiffness matrix — what you actually get

For each part, generation produces a 12×12 **symmetric reduced stiffness matrix** K_{red} , relating the 6 DOF at each of the two reference nodes:

```
DOF order (sorted): [ RP_A: u1 u2 u3 r1 r2 r3 | RP_B: u1 u2 u3 r1 r2 r3 ]
```

Properties to expect and check:

- **Symmetric** ($K = K^T$) — reciprocity.
- **Fully populated** — off-diagonal coupling terms are real (geometry + directional features couple the DOF); do not zero them.
- **Free-free rank deficiency** — the raw generated matrix has **6 zero eigenvalues** (rigid-body modes). This is correct; they vanish once the part is constrained/connected in the global model.
- **Positive semi-definite** in the free state; positive definite once grounded.

Written to an ASCII `.mtx` via `*Substructure Matrix Output, stiffness=YES, output file=USER_DEFINED`; parse it as lower-triangular `(node_i, dof_i, node_j, dof_j, value)` (see the main methodology doc §14.9).

Verification anchor: the axial diagonal term should match $E \cdot A_{\text{net}} / L$ by hand — for the perforated region, $A_{\text{net}} = \text{solid_frac} \cdot A_{\text{gross}}$ with $\text{solid_frac} = 0.683$ (tubesheet) or 0.384 (spool). This is the fastest sanity check that the reduction is behaving.

4. How it is used — the two-job workflow

Substructuring is inherently a **two-phase** process:

Phase 1 — Generation (once per unique part)

1. Mesh the detailed part (holes/waves meshed → **isotropic base metal**; the orthotropy emerges from geometry).
2. Define interfaces; couple each to a reference node (kinematic, since our mating parts are near-rigid).
3. `*Substructure Generate` retaining the interface DOF → produces the superelement `.sim`/`.sup` library and the `.mtx`.
4. (Optional) request a **recovery matrix** scoped to regions where you later want interior stress.

Generation is a **reduction, not a solve** — its ODB is empty. Results appear only in Phase 2.

Phase 2 — Usage (per global model / load case)

1. Import the superelement into the global assembly; place all instances (here, 6× each).
 2. Connect each instance's retained nodes to the surrounding structure (tie / share nodes / connect to neighbors).
 3. Apply global loads and BCs; solve the small reduced system.
 4. (Optional) invoke `(*Substructure Recovery)` to reconstruct interior stress in the recovered regions.
-

5. The specific parts in this assembly

5.1 Tubesheet superelement

- **Detailed content:** hexagonal perforated plate, triangular hole array ($\eta = 0.366$ governing), thickness 2.985", holes meshed, isotropic base metal.
- **Interfaces:** its two mating faces (e.g. top/bottom, or as the assembly dictates), each coupled to a reference node → 12 retained DOF.
- **Reduced object:** a 12×12 stiffness capturing the true perforated-plate compliance, including the in-plane orthotropy the equivalent-solid card only approximates.
- **Recovery:** scope to the hole-ring elements (`(ELS_HOLES)`) if hole stress is needed after the global run; otherwise skip for a lighter superelement.

5.2 Spool superelement

- **Detailed content:** same hole array **plus** horizontal wave cutouts (the flexure feature), thin bridges (~0.05" / 0.199"), fully prismatic, isotropic base metal.
- **Interfaces:** top face → tubesheet (near-rigid); mid-height perimeter strip → foundation (near-rigid). Both kinematic → 12 retained DOF.
- **Reduced object:** a 12×12 stiffness that captures the wave flexure — the whole reason the spool needs a real reduced model rather than a chart-based material card (its in-plane behavior is bending-dominated and cannot be homogenized by ligament efficiency).
- **Why substructure and not homogenization:** the wave terminations vary band to band (irregular edges), so no single homogenized block represents it; the substructure condenses the *actual* geometry.

5.3 The assembled global model

Global assembly (linear static, reduced DOF):

```
6 × tubesheet superelement → 6 × 12 = 72 retained DOF
6 × spool superelement      → 6 × 12 = 72 retained DOF
+ surrounding structure DOF
-----
```

```
Twelve complex parts represented by ~144 reduced DOF total,
from two interior factorizations, reused 6× each.
```

Each tubesheet's top-face reference node ties to the corresponding spool's top-face reference node where they mate; each spool's perimeter reference node ties to the foundation. The global solve moves only the reduced DOF; recovery (if requested) reconstructs hole/bridge stress afterward.

6. To what extent it can be used — scope and limits

Valid for (our case):

- **Linear static** — Guyan/static condensation is exact for linear elastic behavior. ✓ matches the intended analysis.
- **Identical repeated parts** — one generation, many placements. ✓ strongly favorable here.
- **Reusable across load cases** — reduce once, solve many. ✓

Where it stops / needs care:

- **Nonlinearity inside the part.** Static condensation assumes the interior is linear. If a tubesheet or spool were to yield, buckle, or contact internally, the reduced matrix would no longer represent it — you would need the detailed part for those effects (or a nonlinear substructuring scheme, which is far more involved). For **linear static**, not a concern.
- **Dynamics.** Pure Guyan reduction is a static approximation; it is inaccurate for modal/frequency response unless you retain internal modes (Craig-Bampton). Not needed for linear static, but do **not** reuse this same superelement for a modal study without regenerating with retained modes.
- **Interior loads/thermal gradients.** Loads or temperature applied *inside* the reduced region are not represented unless handled at generation (e.g. substructure loads, gravity load cases baked in). For interface-driven mechanical loading, fine.
- **Interface fidelity.** Lumping a face to one reference node with kinematic coupling assumes a rigid interface — valid because the mating parts here are near-rigid. If a mating part were compliant, that assumption would over-stiffen the joint (use distributing coupling or retain the node set instead).

- **Recovery must be pre-planned.** Interior stress is only recoverable if the recovery matrix was requested at generation. It cannot be added afterward.

Net: for 6 identical tubesheets + 6 identical spools under linear static loading, substructuring is well-suited and close to ideal — it is arguably the correct approach rather than merely an optimization.

7. Quick-reference checklist

- ☐ Detailed part meshed with **isotropic** base metal (holes/waves meshed).
 - ☐ Interfaces defined; reference nodes at face centroids; **kinematic** coupling.
 - ☐ `*Retained Nodal Dofs` present (1-6 per RP) — **the bridge to the global model**.
 - ☐ `recovery matrix=YES, elset=ELS_HOLES` **only if** interior stress is needed.
 - ☐ `*Substructure Matrix Output` if you want the readable `.mtx`.
 - ☐ Generation ODB is **expected to be empty** — not a failure.
 - ☐ Generate **once** per unique part; place **6×** in the global model.
 - ☐ Global model: connect retained nodes, apply loads/BCs, solve reduced system.
 - ☐ Verify: axial term vs $E \cdot A_{\text{net}} / L$; matrix symmetry; 6 rigid-body modes free-free; equilibrium of interface reactions.
 - ☐ Recovery (`*Substructure Recovery`) invoked in the global step if hole/bridge stress is wanted.
-

8. Limits of substructuring — stacking, LLA, and nonlinearity

Three questions that define the boundary of the method.

8.1 Can you stack shorter substructures (e.g. four 12" spool segments into a 4' spool)?

Geometrically yes — mechanically, only if you cut through rigid material, which the spool usually violates.

Stacking substructures by tying the retained interface nodes of one to the next is a legitimate technique **for a uniform column** (a plain pipe, a lattice tower): identical segment generated once, tied end-to-end N times. It is efficient and accurate *when each cut plane passes through a section that genuinely behaves like a rigid 6-DOF interface*.

The spool breaks that assumption. Its compliance is the **wave flexure**, and the waves run **horizontally — across the through-thickness (stacking) direction**. If you slice the spool into 12" segments, each cut plane may pass **through a wave band**, and then you lump that internal cross-section to a single reference node with kinematic coupling. That **rigidizes a**

section that is actually compliant, artificially stiffening the very flexure you are trying to represent. Four stacked rigid-interface segments $\neq$ one continuous flexure. It is the §2.2 error (kinematic on a compliant interface) relocated to an internal cut.

Decision rule:

- **Waves only in discrete bands, solid material between them** $\rightarrow$ you *may* stack, but cut **only through solid sections**, never through a wave band. Place segment boundaries in the rigid ligaments.
- **Waves run the full height** $\rightarrow$ **generate the spool as one full-height superelement**. It is prismatic (swept mesh, §12 of the methodology doc), so meshing the full 4' is cheap; segmenting saves little and loses fidelity.
- **When in doubt** $\rightarrow$ one full-height superelement is the safe choice. You lose the modest reuse benefit but keep the flexure exact.

The determining factor is simply *where the compliance lives relative to the cut*.
Stacking is safe across rigid cuts, unsafe across compliant ones.

8.2 Can you run a Limit Load Analysis on a substructure? — No.

LLA is **elastic-perfectly-plastic**: the ligaments yield and the load redistributes. A static-condensed substructure is **linear by construction** — the reduction $\overline{K_{red} = K_{rr} - K_{ri} K_{ii}^{-1} K_{ir}}$ bakes in the assumption that the interior stays elastic. Condensation freezes the interior as a linear spring; its ability to yield is gone.

So the perforated region *inside* a superelement **cannot participate in a plastic collapse**. LLA must run on the **detailed, fully-meshed part with EPP material** — which is exactly why the methodology doc keeps LLA (§9–§10) and substructuring (§2, §14) on separate tracks. You can use a superelement for the *linear* stiffness of a neighbor while running LLA on the detailed part of interest, but you cannot collapse a condensed region.

8.3 Can you run nonlinear analysis using a substructure? — Qualified.

- **Nonlinearity OUTSIDE the reduced part: allowed**. A superelement can sit inside a globally nonlinear model — global contact between parts, other parts yielding, large-rotation geometric nonlinearity of the assembly. Throughout, the superelement contributes its **constant linear stiffness**, correctly, as long as *it* stays linear.
- **Nonlinearity INSIDE the reduced part: not represented**. The condensed interior cannot yield, buckle, or develop internal contact. If the tubesheet's own ligaments must yield, or the spool's bridges must buckle, you need the **detailed** part for those effects.

Rule of thumb: the substructure is a linear black box. The world around it can be as nonlinear as you like; its insides cannot change. For the intended **linear static** global

analysis, all of this is moot — you are squarely in the valid regime.

9. Reading your first stiffness matrix — worked example

Using the 3-hole simplified-tubesheet superelement generated per §14.

9.1 Layout

```
*USER ELEMENT, NODES=2, LINEAR
*MATRIX, TYPE=STIFFNESS
```

- **2 nodes × 6 DOF = a 12 × 12 matrix**, written **lower-triangular** (row *i* has *i* entries).
- **DOF order:** [node1: u1 u2 u3 r1 r2 r3 | node2: u1 u2 u3 r1 r2 r3] → global indices 1-6 (node 1), 7-12 (node 2).
- The two nodes are your two retained reference nodes.

9.2 The diagonal terms (direct stiffnesses)

Index	DOF	Value	Node-2 twin
1	node1 u1	2.355e7	7: 2.355e7
2	node1 u2	5.397e6	8: 5.397e6
3	node1 u3	3.770e6	9: 3.770e6
4	node1 r1	2.616e6	10: 2.616e6
5	node1 r2	5.772e6	11: 5.773e6
6	node1 r3	1.439e7	12: 1.439e7

First health check — passed. Node 1's six diagonals equal node 2's six diagonals (2.355e7 = 2.355e7, 3.770e6 = 3.770e6, ...). For a symmetric part the two ends must have identical direct stiffness. ✓

9.3 It behaves like a clean two-node spring

The node1-node2 coupling block mirrors the diagonals with a minus sign:

$$K(7,1) = -2.355e7 = -K(1,1)$$

$$K(8,2) = -5.397e6 = -K(2,2)$$

$$K(9,3) = -3.770e6 = -K(3,3)$$

That is the textbook $\begin{bmatrix} k & -k & \\ -k & k & \end{bmatrix}$ spring structure for each translational DOF: pushing the two nodes together is resisted, uniform translation costs nothing. ✓ This is exactly what a healthy 2-node reduced stiffness looks like.

9.4 Which DOF is which physical direction

Do not assume — read it off your RP coordinates. The axis along which your two reference nodes are **separated** is your **extension (pull) axis**; its diagonal is the extension stiffness (normally the largest of the three translational terms). The other two translational diagonals are **transverse shear**.

Here the translational terms are **2.355e7 (DOF1) » 5.40e6 (DOF2) > 3.77e6 (DOF3)**. Since extension is stiffer than shear ($E > G$), **DOF 1 is almost certainly your extension axis** — meaning your two RPs are separated along **X**. DOF 2 and DOF 3 are the two shear directions.

Correction to an earlier note: if you had separated the RPs along Z (through-thickness), DOF 3 would be the extension term and would be the *largest* translational diagonal. It is not — DOF 1 is. So confirm your RP separation axis; it is X here, not Z. The verification formula below applies to whichever axis is your extension direction.

9.5 Hand-check the extension term

For the extension DOF (here DOF 1 = 2.355e7):

$$\begin{aligned} K_{\text{ext}} &\approx E_m \cdot A_{\text{net}} / L \\ &= E_m \cdot (\text{solid_frac} \cdot A_{\text{gross}}) / L \end{aligned}$$

along the extension axis, where L = the span between your two RP faces along that axis, A_{net} = net (solid) cross-section perpendicular to it. Plug in your simplified block's dimensions, $E_m = 27e6$, and the solid fraction of the 3-hole patch. If 2.355e7 lands within a few percent (slightly below the ideal), the reduction is behaving. **This single check is the most valuable thing to confirm on a first build.**

9.6 Real coupling vs. numerical noise

The matrix has two very different classes of off-diagonal term — do not treat them the same:

- **Genuine couplings (order 1e5-1e6):** e.g. $K(5,3) \approx -3.77e6$, $K(6,2) \approx 5.40e6$. These are **real lever-arm couplings** from the rigid kinematic coupling — rotating the

reference node produces translation at the offset face. Keep them; they carry the bending/torsion behavior.

- **Numerical noise (order 1e-3-1e-4):** e.g. $K(3,1) \approx 6e-4$, $K(4,1) \approx 1e-4$. These sit ~10 orders of magnitude below the real terms and represent couplings that are **zero by symmetry** (in-plane translation does not couple to the perpendicular in-plane translation). Read them as zero — they are round-off, not physics.

$K(2,1) \approx 163$ is likewise negligible against $2.4e7 \rightarrow$ the two in-plane translations are effectively uncoupled.

9.7 The definitive check — rigid-body modes

Run the §14.9.4 MATLAB parser on this `.mtx` and take the eigenvalues. The raw generated matrix is **unconstrained (free-free)**, so it must have **exactly 6 near-zero eigenvalues** — the 6 rigid-body modes (3 translation, 3 rotation). The `[k, -k; -k, k]` structure already guarantees the 3 translational ones.

- **Exactly 6 near-zero $\rightarrow$ healthy.** They vanish once you fix/connect a node in the global model.
- **Fewer than 6 $\rightarrow$ over-constrained** (a spurious internal restraint, or a coupling error).
- **More than 6 $\rightarrow$ a mechanism** (a retained DOF left unsupported, or a bad coupling).

This is the single most reliable health check on any generated superelement — more definitive than any individual term.

9.8 Verdict on this matrix

Node-block symmetry holds, the two-node spring structure is clean, the couplings are the expected lever-arm terms, and the noise terms are properly negligible. It looks like a correctly generated superelement. Two things left to close it out: (1) confirm which axis your RPs are separated along and hand-check that extension term against $E \cdot A_{\text{net}}/L$, and (2) run the eigenvalue check for exactly 6 rigid-body modes.